TE Artificial Intelligence and Soft Computing
Mini Project Report

On

AegisOps: A Demonstration System for AI-Powered Cloud and Kubernetes Monitoring and Self-Healing

Submitted in partial fulfillment of the requirement of
University of Mumbai for the Degree of

Bachelor of Engineering

in

Artificial Intelligence and Data Science.

Submitted By

Nihal Mishra

Supervisor

Prof Nirosha Uppu



Department of Artificial Intelligence and Data Science.

Smt. Indira Gandhi College of Engineering, Ghansoli – 400701

An Autonomous Institute
Academic Year 2026 – 27

Department of Artificial Intelligence and Data Science.
SMT. INDIRA GANDHI COLLEGE OF ENGINEERING
GHANSOLI – 400701 (An Autonomous Institute)

CERTIFICATE

This is to certify that the requirements for the TE Mini Project report entitled '**AegisOps: A Demonstration System for AI-Powered Cloud and Kubernetes Monitoring and Self-Healing**' have been successfully completed by the following students:

Name	**Roll No.**
Nihal Mishra	33

in partial fulfillment of Bachelor of Engineering in Department of Artificial Intelligence and Data Science, Smt. Indira Gandhi College of Engineering, Ghansoli 401701, An Autonomous Institute during the Academic Year 2026 – 2027.

Prof Nirosha Uppu
Supervisor

Dr. Shankar M. Patil
Head of Department

Dr. Sunil Chavan
Principal



Department of Artificial Intelligence and Data Science

SMT. INDIRA GANDHI COLLEGE OF ENGINEERING
GHANSOLI – 400701 (An Autonomous Institute)

REPORT APPROVAL

This TE Mini Project report entitled "**AegisOps: A Demonstration System for AI-Powered Cloud and Kubernetes Monitoring and Self-Healing**" by **Nihal Mishra** is approved for the degree of **Bachelor of Engineering** Department of Artificial Intelligence and Data Science, **Smt. Indira Gandhi College of Engineering, Ghansoli.** (An Autonomous Institute)

Examiners:

1. _____

2. _____

Supervisors:

1. _____

2. _____

Chairman:

1. _____

Date:

Place:

Declaration

We declare that this written submission for TE Mini Project Report Declaration entitled "**AegisOps: A Demonstration System for AI-Powered Cloud and Kubernetes Monitoring and Self-Healing**" represents our ideas in our own words and where others' ideas or words have been included. We have adequately cited and referenced the original sources. We also declared that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any ideas / data / fact / source in our submission. We understand that any violation of the above will cause disciplinary action by the institute and also evoke penal action from the sources which have thus not been properly cited or from whom paper permission has not been taken when needed.

Project Group Members:

Student 1 Name & Sign: _____

Student 2 Name & Sign: _____

Student 3 Name & Sign: _____

Student 4 Name & Sign: _____

Date:

Place:

Table of Contents

Abstract i

List of Figures ii

List of Tables iii

1. Introduction 1

1.1 Fundamentals 1

1.2 Objectives 2

1.3 Scope 2

1.4 Outline 3

2. Literature Survey 3

2.1 Machine Learning and AIOps 4

2.1.1 AIOps for Failure Management 4

2.1.2 Anomaly Detection with Isolation Forest 4

2.1.3 Random Forest for Prediction and Classification 4

2.2 Soft Computing, Reasoning and Language Techniques 4

2.2.1 Fuzzy Logic and Rule Inference 5

2.2.2 Declarative Policy Rules 5

2.2.3 Retrieval-Augmented Generation and Small Language Models 5

2.3 Literature Summary 5

3. Proposed System 7

3.1 Overview 7

3.1.1 Existing System Architecture 7

3.1.2 Proposed System Architecture 8

3.2 Implementation Details 9

3.2.1 Algorithm / Tools / Techniques 9

3.2.2 Use Case / Activity Diagram 10

3.2.3 Dataset and Input Details 11

3.2.4 Hardware and Software Specifications 12

4. Performance and Results 14

4.1 Dashboard and Module Results 14

4.2 Comparative Results 19

4.3 Problems Found and Fixed During Testing 23

5. Applications 25

5.1 Social 25

5.1.1 Learning AIOps Safely 25

5.1.2 Reliable Public Digital Services 25
5.1.3 Reduced On-Call Fatigue 25
5.2 Technical 25
5.2.1 Incident Response and Root Cause Analysis 25
5.2.2 Predictive Operations and CI/CD Risk 25
5.2.3 Governed Automation and Cost Awareness 25
6. Conclusion and Future Scope 26
6.1 Conclusion 26
6.2 Future Scope 26
References 27
Acknowledgement 28

Abstract

Modern cloud applications run as many small services on Kubernetes. Each service produces a stream of telemetry, and an on-call engineer must notice a fault, find its cause, decide what to do and do it safely, often at night and under time pressure. Conventional monitoring tools draw graphs and fire alerts when a fixed threshold is crossed. They raise many false alarms, they do not predict failures, they do not explain the cause in plain language, and they leave the fix to a human. Artificial Intelligence for IT Operations (AIOps) tries to close this gap by adding learning, reasoning and automation to monitoring.

In this report we design and implement AegisOps, a demonstration system for AI-powered monitoring and self-healing of cloud and Kubernetes workloads. AegisOps is a demo build: the telemetry is synthetic, the Kubernetes cluster is simulated and the language model is replaced by a deterministic template, so no real infrastructure is touched. It follows one lifecycle: observe, understand, predict, investigate, explain, decide, act and verify. An Isolation Forest detects anomalous telemetry, a Random Forest predicts the probability that a service will fail, a second Random Forest classifies service health as healthy, degraded or critical, and a fuzzy rule engine turns CPU, memory, latency, error rate and pod restarts into an overload risk score. A root cause analysis engine ranks causes using metric, deployment and service-dependency evidence. Runbooks are retrieved for the explanation, and every proposed action passes a declarative policy engine that decides ALLOW, REQUIRE_APPROVAL or DENY for the user role and risk level. Approved actions scale a deployment, restart a pod or roll back a release, recovery is verified from the service telemetry, and every execution is written to an audit log. The system has a FastAPI backend, a React dashboard updated over a WebSocket, a failure simulator and an optional adapter for a live cluster.

On held-out synthetic telemetry the Random Forest failure predictor reached 94.1 % accuracy with a ROC-AUC of 0.989, the service health classifier reached 99.9 %, and the Isolation Forest separated 950 normal samples from 50 anomalous ones without error. The fuzzy engine scored a CPU saturation fault at 1.00 (critical risk) and an idle service at 0.15 (low risk). In the end-to-end tests a CPU saturation incident on the checkout service was detected, analysed and resolved by a policy-approved scale-out to four replicas, which brought CPU from 94.5 % to 37.8 % and latency from 420 ms to 126 ms in the simulation; a memory leak and a pod crash loop were resolved by restarts, a rollback was approved and executed by a DevOps user, and a Viewer was denied. A whole incident, from trigger to verified recovery, takes about 0.13 s on the server, and all 18 unit tests pass. The report also lists the problems found and fixed while testing, and the limits that remain, such as the use of synthetic data and a simulated cluster.

List of Figures

Fig 1.1	AegisOps Pipeline	1
Fig 2.1	Membership functions of the overload-risk engine for CPU, memory and latency	5
Fig 3.1	AegisOps AI and Soft Computing core taxonomy	7
Fig 3.2	Existing system architecture used for threshold-based monitoring	8
Fig 3.3	Proposed AegisOps system architecture	8
Fig 3.4	Activity diagram of one incident from detection to verified recovery	11
Fig 4.1	Overview page at the healthy baseline	14
Fig 4.2	Overview page during a CPU saturation incident on checkout-service	15
Fig 4.3	Infrastructure page: nodes and pods	15
Fig 4.4	AI Intelligence page: live model outputs and fuzzy rule degrees	16
Fig 4.5	Incident Center: root cause, evidence, explanation and retrieved runbooks	17
Fig 4.6	Remediation page for the Viewer role: action denied by policy	17
Fig 4.7	Remediation page for the DevOps role: executed and verified	18
Fig 4.8	Overview page after verified recovery	18
Fig 4.9	Copilot answering from live telemetry (demo template)	19
Fig 4.10	Cloud Cost page with an anomalous cost line	19
Fig 4.11	Failure predictor: confusion matrix and feature importance	20
Fig 4.12	Isolation Forest scores of normal and anomalous telemetry	20
Fig 4.13	Overload risk score against CPU and memory	21

List of Tables

Table 1.1	Mapping of AI and Soft Computing Subsystems in AegisOps	2
Table 2.1	Summary of literature survey	6
Table 3.1	Sample Dataset Used for Experiment	12
Table 3.2	Hardware details	12
Table 3.3	Software details	13
Table 4.1	Performance of the machine learning models	20
Table 4.2	Fuzzy overload-risk engine on test cases	21
Table 4.3	Root cause analysis on test cases	22
Table 4.4	Policy decisions for 32 role, action and risk combinations	22
Table 4.5	End-to-end incident tests	23
Table 4.6	Measured response times	23
Table 4.7	Problems found during testing and their fixes	24

Chapter 1
Introduction

1.1 Fundamentals

A modern online service is rarely one program. It is a set of small services, such as checkout, payment, authentication and inventory, packaged in containers and run by Kubernetes on a cluster of machines. Kubernetes starts the containers, restarts them when they crash and can add more copies when asked. But it does not know why a service is slow, whether it is about to fail, or which action is safe. That knowledge lives in the head of the on-call engineer. When CPU usage climbs, latency rises and errors appear, the engineer opens several dashboards, searches logs, remembers that a new version was deployed an hour earlier, finds the runbook and finally scales the deployment or rolls back the release.

AegisOps demonstrates how this chain of work can be automated while a human stays in control of risky steps. It watches the telemetry of every service (CPU, memory, latency, error rate and pod restarts) and applies Artificial Intelligence and Soft Computing techniques at each stage. An **Isolation Forest** finds telemetry that does not look normal. A **Random Forest** estimates the probability that the service will fail and names the metrics that contribute most. A second Random Forest labels the service healthy, degraded or critical. A **fuzzy inference** engine combines the metrics into a smooth overload risk score. A **root cause analysis** engine collects evidence from metrics, recent deployments and the service dependency graph and ranks the probable causes. **Runbook retrieval** supplies the explanation, a **declarative policy engine** decides whether the proposed fix is allowed, needs approval or is denied, and the **remediation engine** performs the scale-out, restart or rollback and verifies recovery.

Two design rules shape the system. First, the AI only proposes: every action goes through the policy engine, and a denied action cannot be executed. Second, AegisOps is a **demo build**. With `DEMO_MODE` on (the default) the cluster is simulated, so scaling, restarting and rolling back only change the demo database, and the explanation comes from a deterministic template instead of a language model. The failure simulator injects CPU saturation, memory leaks, pod crash loops, traffic surges, network latency and cascading failures so that the whole workflow can be shown without any real infrastructure.

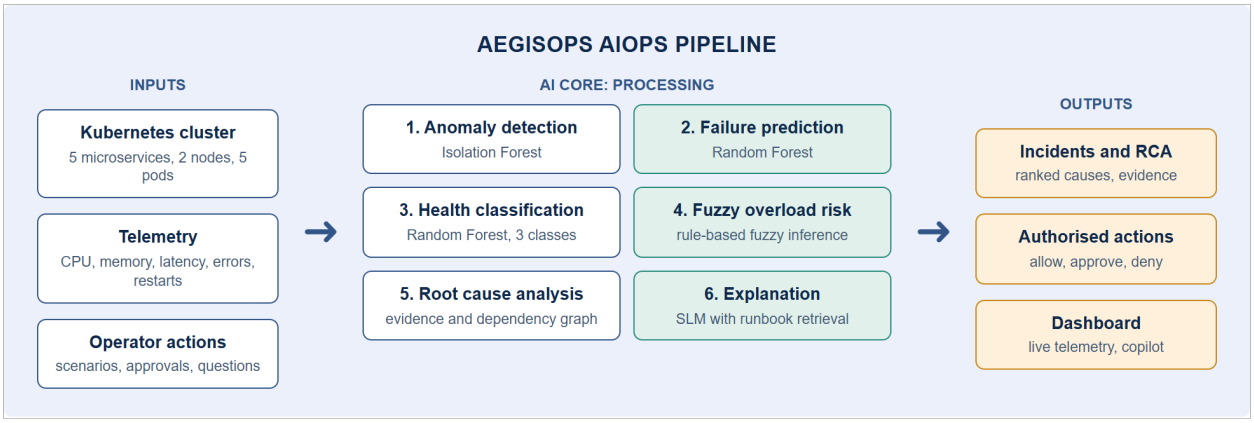


Fig 1.1: AegisOps Pipeline

1.2 Objectives

Table 1.1 summarises the AI and Soft Computing techniques integrated in AegisOps together with their roles in the incident lifecycle.

Subsystem	AI / Soft Computing Technique	Primary Objective in the System
Anomaly detection	Isolation Forest (unsupervised)	Flag telemetry that differs from normal behaviour without needing labelled faults.
Failure prediction	Random Forest classifier	Estimate the probability of failure and list the top contributing metrics.
Health classification	Random Forest classifier (3 classes)	Label each service HEALTHY, DEGRADED or CRITICAL in real time.
Overload risk	Fuzzy rule inference	Turn CPU, memory, latency, errors and restarts into a smooth risk score from 0 to 1.
Root cause analysis	Evidence rules and dependency graph	Rank probable causes from metric, deployment and dependency evidence.
Explanation	Runbook retrieval and a template (language model optional)	Explain the incident in plain language and suggest a fix.
Authorisation	Declarative role, action and risk rules	Decide ALLOW, REQUIRE_APPROVAL or DENY for each proposed action.
Remediation	Simulated Kubernetes scale, restart and rollback	Act on approved fixes, verify recovery and keep an audit log.

Table 1.1: Mapping of AI and Soft Computing Subsystems in AegisOps

The objectives of this work are as follows:

1. To study threshold-based monitoring and manual incident response and identify their limitations, in order to propose an AI-assisted monitoring and self-healing workflow for Kubernetes workloads.
2. To build a working demonstration, with a backend, a database, a live dashboard, a failure simulator and a simulated Kubernetes adapter, that follows the lifecycle of observing, understanding, predicting, investigating, explaining, deciding, acting and verifying.
3. To implement and evaluate an Isolation Forest, two Random Forest models, a fuzzy overload-risk engine, a root cause analysis engine and a policy engine, and to measure them with accuracy, precision, recall, F1 score, ROC-AUC, confusion matrices, unit tests and end-to-end incident tests.

1.3 Scope

AegisOps is scoped as a demonstration for students, instructors and presentations of intelligent operations; it has no real functionality beyond the demo. The deliverables are a FastAPI backend with REST and WebSocket interfaces, a SQLAlchemy database (SQLite by default), a React and Vite dashboard with seven pages, a failure simulator with six scenarios, machine learning and fuzzy models, a root cause analysis engine, a runbook retrieval and explanation module, a policy engine, a simulated Kubernetes adapter and automated tests.

All telemetry is synthetic. The models are trained on generated data whose labels follow a fixed rule, so the reported accuracy shows how well the models recover that rule and not how they would perform on a production cluster. In demo mode the Kubernetes adapter never contacts a cluster and recovery after an action is simulated by updating the service record and re-reading it; the language model (Ollama) is not called. A live-cluster option and an Ollama option exist in the code for scale and restart, but they were not used in the experiments. Training on real telemetry from Prometheus, a real Prolog policy engine and vector-based retrieval are listed as future work.

1.4 Outline

The report is organised as follows. Chapter 1 introduces the problem, the objectives and the scope. Chapter 2 presents the literature survey of anomaly detection, ensemble learning, fuzzy inference, rule-based policy, retrieval-augmented language models and AIOps. Chapter 3 describes the existing and proposed architectures, the algorithms, the datasets, and the hardware and software used. Chapter 4 presents the performance and results of the experiments, the problems found and fixed during testing, and the remaining limits. Chapter 5 discusses social and technical applications, and Chapter 6 concludes the report and lists future work.

Chapter 2
Literature Survey

2.1 Machine Learning and AIOps

The techniques in this category learn what normal behaviour looks like, or learn a mapping from telemetry to a failure label. They need data but little hand-written logic. Russell and Norvig [1] describe the general framework of learning agents that this section follows.

2.1.1 AIOps for Failure Management

Notaro, Cardoso and Gerndt [10] survey AIOps methods for failure management and group them into failure prediction, failure detection, root cause analysis and remediation. Site Reliability Engineering [9] describes the practices that these methods automate: monitoring, alerting on symptoms, incident response and postmortems. Kubernetes [11] provides the declarative control plane that restarts and scales containers, which makes the scale and restart actions in AegisOps possible. AegisOps implements all four failure-management tasks in one lifecycle and connects them to the same set of services.

2.1.2 Anomaly Detection with Isolation Forest

Liu, Ting and Zhou [2] proposed the Isolation Forest, which isolates an observation by random splits of the feature space. Anomalies are few and different, so they are isolated after fewer splits than normal points. The anomaly score of a sample x in a set of n samples is

$$s(x, n) = 2^{-E[h(x)] / c(n)} \tag{2.1}$$

where E[h(x)] is the average path length over the trees and c(n) is the average path length of an unsuccessful search in a binary tree. The method needs no labelled faults, runs in time that grows as n log n and needs no GPU, which suits continuous monitoring. In AegisOps (`isolation_forest.py`) the scikit-learn implementation [8] is trained on 1000 normal-looking telemetry vectors with 100 trees and a contamination of 5 %. The decision is the forest own prediction, and the 0 to 1 score reported to the operator is calibrated so that 0.5 lies on the decision threshold (Equation 3.1).

2.1.3 Random Forest for Prediction and Classification

Breiman [3] introduced random forests, ensembles of decision trees trained on random samples of the data and features, whose votes are combined. They resist over-fitting, handle features of different scales and give a feature importance ranking. AegisOps uses two forests of 100 trees: one predicts whether a service is about to fail (7 features) and one classifies health into three classes (5 features). The probability is the fraction of trees voting for failure, and the importance ranking is shown to the operator as the top contributing features.

2.2 Soft Computing, Reasoning and Language Techniques

The techniques in this category handle imprecise inputs, encode expert rules, or use language.

2.2.1 Fuzzy Logic and Rule Inference

Zadeh [4] introduced fuzzy sets, in which an element has a degree of membership between 0 and 1. Mamdani and Assilian [5] applied fuzzy rules to control: inputs are fuzzified, each rule fires with the minimum of its conditions and the results are combined and converted to one number. The overload-risk engine of AegisOps (`overload_risk.py`) describes CPU, memory, latency, error rate and pod restarts with low, medium and high sets (Figure 2.1), fires rules with minimum and maximum operators, and combines the rule degrees into a score with a weighted average of fixed output levels. The advantage is a smooth, readable score; the limitation is that the membership functions and rule weights are set by hand and are not learned.

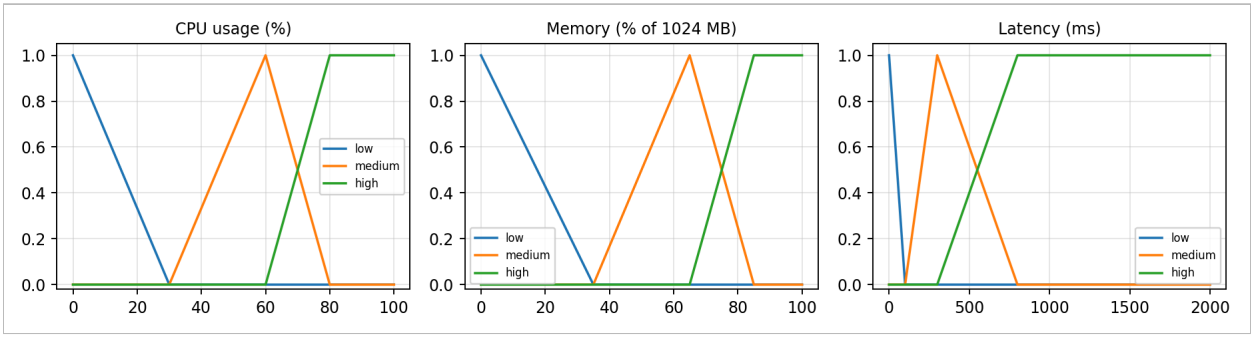


Fig 2.1: Membership functions of the overload-risk engine for CPU, memory and latency

2.2.2 Declarative Policy Rules

Kowalski [7] showed that logic can be used to state what is permitted and let the machine derive the consequences. A policy is naturally a set of facts of the form rule(Role, Action, Risk) giving a Decision. The policy engine of AegisOps (`policy_engine.py`) holds exactly this table for four roles, four action types and two risk levels and prints the matched rule, for example `rule(DevOps, SCALE_DEPLOYMENT, LOW) -> ALLOW`. It is Prolog-style and is implemented as a table in Python, not as a Prolog program, which keeps it fast and free of external dependencies. Separating the decision from the AI that proposes an action prevents a model from acting outside its authority.

2.2.3 Retrieval-Augmented Generation and Small Language Models

Lewis et al. [6] showed that a language model answers more reliably when relevant documents are retrieved and placed in its prompt. Small instruction-tuned models such as the Qwen2.5 family [12] can run on a laptop through a local server such as Ollama, which keeps operational data on the premises. AegisOps retrieves the two runbooks that best match the incident (by word overlap) and builds the explanation from the root cause, the evidence and the runbook. In the demo build the explanation is produced by a deterministic template; the code can send the same prompt to an Ollama model when `DEMO_MODE` is turned off.

2.3 Literature Summary

A literature review is an objective, critical summary of published research relevant to a topic. The summary of the literature used to build AegisOps is presented in Table 2.1.

SN	Paper / Author	Advantages and Disadvantages
1.	P. Notaro et al. [10], B. Beyer et al. [9] (AIOps, SRE)	**Advantages:** A clear taxonomy of prediction, detection, RCA and remediation. **Disadvantages:** Surveys do not give an integrated, working pipeline.
2.	F. T. Liu et al. [2] (Isolation Forest)	**Advantages:** Unsupervised, fast, no GPU. **Disadvantages:** Scores need calibration; a threshold must be chosen.
3.	L. Breiman [3] (Random Forest)	**Advantages:** Accurate, robust, gives feature importance. **Disadvantages:** Needs labelled data; predicts only what the labels describe.
4.	L. A. Zadeh [4], E. Mamdani and S. Assilian [5] (Fuzzy inference)	**Advantages:** Smooth, readable risk scores from imprecise inputs. **Disadvantages:** Hand-set membership functions do not adapt.
5.	R. Kowalski [7] (Logic-based rules)	**Advantages:** Policies are explicit, auditable and testable. **Disadvantages:** Rules must be written and maintained by people.
6.	P. Lewis et al. [6], Qwen2.5 [12] (RAG, small language models)	**Advantages:** Plain-language explanations grounded in runbooks, running locally. **Disadvantages:** Can hallucinate, so it must not act without a policy check.
7.	B. Burns et al. [11] (Kubernetes)	**Advantages:** Declarative scaling and self-restart. **Disadvantages:** No notion of why a service is unhealthy.

Table 2.1: Summary of literature survey

Chapter 3
Proposed System

3.1 Overview

The system overview of **AegisOps** is presented in this section. The classification of the AI and Soft Computing techniques integrated in the cloud-operations domain is shown in Figure 3.1.

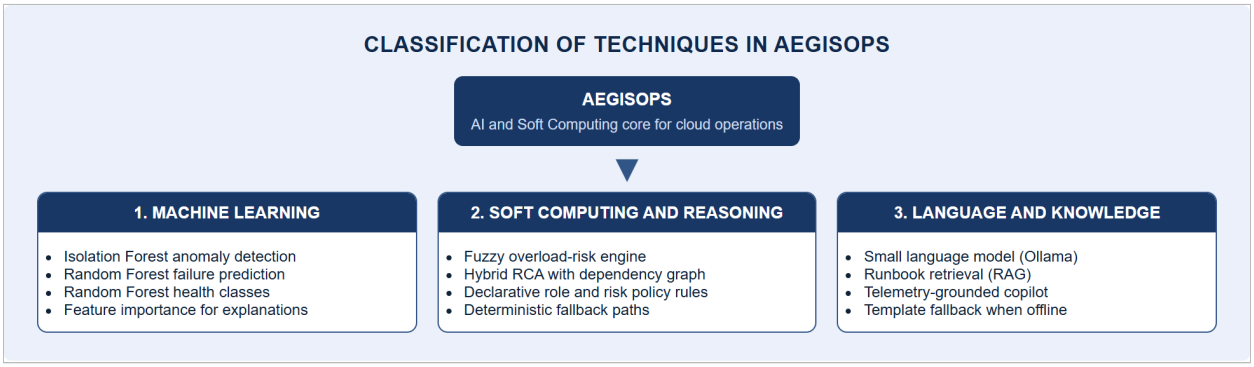


Fig 3.1: AegisOps AI and Soft Computing core taxonomy

The system helps operators and students see how AI techniques cooperate during one incident. Based on the nature of the problem each technique solves, the techniques fall into the following paradigms:

- **Machine Learning:** The Isolation Forest (`isolation_forest.py`) detects anomalous telemetry, the failure predictor (`failure_predictor.py`) estimates failure probability and the service classifier (`service_classifier.py`) labels health.
- **Soft Computing and Reasoning:** The fuzzy overload-risk engine (`overload_risk.py`) gives a continuous risk score, the RCA engine (`rca_engine.py`) ranks causes, and the policy engine (`policy_engine.py`) authorises actions.
- **Language and Knowledge:** The assistant (`slm_assistant.py`) and the retrieval engine (`rag_engine.py`) explain incidents and answer questions in the copilot page; in the demo build the text comes from a template.
- **Automation:** The remediation engine (`remediation_engine.py`) and the Kubernetes adapter (`k8s_client.py`) scale deployments, restart pods and roll back releases in simulation and record an audit log.

3.1.1 Existing System Architecture

Conventional cloud monitoring operates as a collector, a set of graphs and a set of static alert rules. It lacks a model of normal behaviour, does not predict, and cannot explain or act.

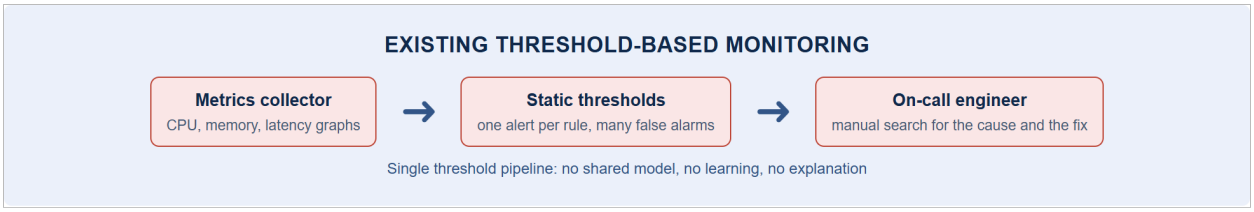


Fig 3.2: Existing system architecture used for threshold-based monitoring

The high-level architecture of an existing system is depicted in Figure 3.2. Its operation has three rigid steps:

1. **Metrics collector:** Gathers CPU, memory and latency series from the cluster and draws graphs.
2. **Static thresholds:** Raises an alert whenever one metric crosses a fixed value. A short spike causes a false alarm, while a slow multi-metric problem may cause none.
3. **On-call engineer:** Reads the alert, searches logs, deployments and runbooks by hand, decides on the fix and applies it. The time to recover depends on the engineer.

3.1.2 Proposed System Architecture

To overcome these limitations AegisOps places a FastAPI backend at the centre and connects the AI components to it, as shown in Figure 3.3. The backend stores services, nodes, pods, incidents, predictions, runbooks and audit records in a database, advances the simulation every two seconds, runs the models on the current telemetry, and broadcasts the result over a WebSocket (`/ws/monitoring`) to the React dashboard.

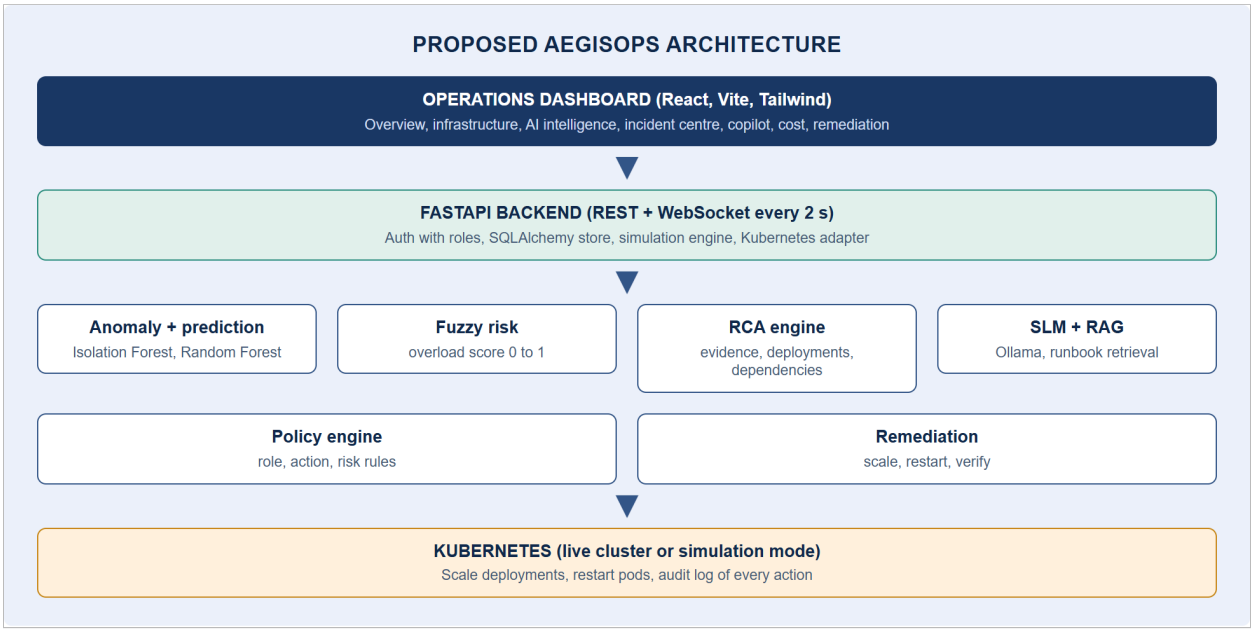


Fig 3.3: Proposed AegisOps system architecture

The components are combined using the following strategies:

- **Layered analysis:** The same telemetry vector is passed to the anomaly detector, the failure predictor, the health classifier and the fuzzy engine, so the operator sees four complementary views of one service. The dashboard tiles show these live outputs for the worst service.

- **Evidence before explanation:** The RCA engine produces ranked causes and evidence first; the explanation only turns that structured result into text. Metric evidence outranks a recent deployment, which is listed as a contributing factor.
- **Policy gate:** Every proposed action becomes a remediation record whose status (APPROVED, PENDING_APPROVAL or DENIED) comes from the policy engine for the selected role and risk. A denied action cannot be executed, and an action that needs approval can be approved only by an Admin or DevOps user.
- **Verified recovery:** After an action the service record is re-read and compared with healthy limits; the incident is resolved only if recovery is verified.
- **Demo mode:** Kubernetes operations and the language model are simulated, so the workflow runs offline and answers in milliseconds.

3.2 Implementation Details

The implementation details and the step-by-step procedure of AegisOps are given in this section.

3.2.1 Algorithm / Tools / Techniques

Algorithm: Handling One Incident from Detection to Verified Recovery

Input: Telemetry of a service (CPU, memory, latency, error rate, pod restarts), recent deployments, the user role. **Output:** Ranked root causes, an explanation, a policy decision, an executed action and the incident status.

1. **Step 1 (Detect):** Build the feature vector and compute the anomaly decision and calibrated score, the failure probability with its top three features, the health class and the fuzzy overload risk.
2. **Step 2 (Open incident):** When a CPU, memory or crash scenario is started or a fault is detected, create an incident with a unique code, severity and the affected service.
3. **Step 3 (Investigate):** The RCA engine adds metric evidence (CPU above 85 %, memory above 512 MB, more than two restarts, error rate above 5 %), deployment evidence for the service or its dependencies, and dependency-propagation hypotheses. A metric-based cause has confidence 0.85 to 0.88 and outranks a deployment (0.75 as a contributing factor); a deployment is the primary cause (0.90) only when no metric explains the incident.
4. **Step 4 (Explain):** Retrieve the two best-matching runbooks, build the explanation, and map the root cause to a suggested action (scale for CPU, restart for memory or crash loops, rollback for a deployment).
5. **Step 5 (Authorise):** Look up rule(Role, Action, Risk) in the policy table. Apply the safety override for protected resources. The decision is ALLOW, REQUIRE_APPROVAL or DENY.
6. **Step 6 (Act):** Refuse a denied action and refuse an approval by a role other than Admin or DevOps; otherwise call the (simulated) Kubernetes adapter to scale, restart or roll back.
7. **Step 7 (Verify and learn):** Re-read the service telemetry, compare it with healthy limits (CPU below 60 %, latency below 200 ms, errors below 2 %, memory below 450 MB), set the status SUCCESS or

UNVERIFIED, resolve the incident on success, and write an audit log entry with the user, role, action and result.

$$score_{cal} = clip(0.5 + (offset - s) / 0.05, 0, 1) \quad (3.1)$$

Equation (3.1) gives the calibrated anomaly score, where s is the raw Isolation Forest score of the sample and offset is the decision threshold learned by the forest. A score of 0.5 lies on the threshold, normal telemetry scores near 0 and strongly abnormal telemetry near 1. A sample is an anomaly when the forest itself predicts one, and the severity is MEDIUM, HIGH (above 0.65) or CRITICAL (above 0.80).

$$score = 0.95\ c + 0.75\ h + 0.50\ m + 0.15\ (1 - max(c, h, m)) \quad (3.2)$$

Equation (3.2) is the defuzzification used by the overload-risk engine, where c is the degree of the critical rules (CPU, memory and latency all high, or CPU and error rate both high), h the degree of the high rules (CPU and memory high, memory and latency high, memory high alone, latency and errors high, or restarts and errors high) and m the degree of the medium rule (CPU, memory and latency all medium). The score is limited to the range 0 to 1 and is classified as LOW (below 0.35), MEDIUM (0.35 to 0.60), HIGH (0.60 to 0.80) or CRITICAL RISK (0.80 and above).

Tools: The backend is written in Python 3.11 with FastAPI, Pydantic, SQLAlchemy and Uvicorn. The models use scikit-learn, NumPy and joblib. The frontend uses React 18, Vite, Tailwind CSS and Recharts. Tests are written with unittest and run with pytest.

3.2.2 Use Case / Activity Diagram

Figure 3.4 shows the activity diagram of one incident. Each block has a dedicated role:

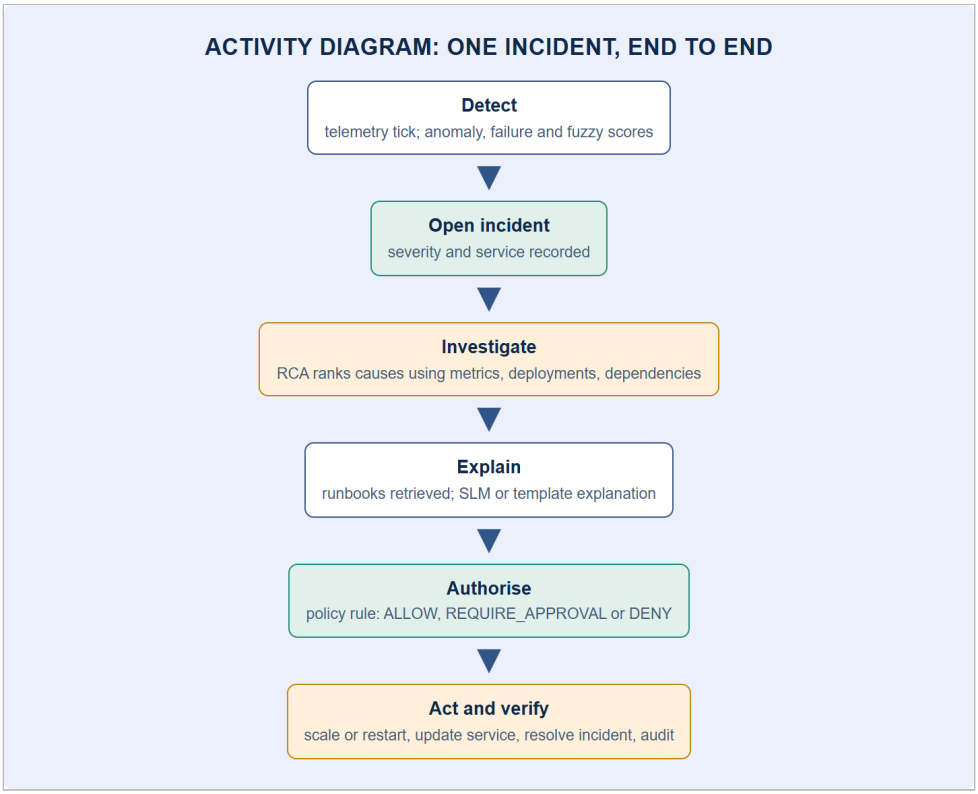


Fig 3.4: Activity diagram of one incident from detection to verified recovery

1. **Intelligence Block (`ml/`):** Provides anomaly detection, failure prediction, health classification and the fuzzy overload score.
2. **Investigation Block (`rca_engine.py`, `slm_assistant.py`, `rag_engine.py`):** Ranks causes from evidence and writes the explanation with runbook references.
3. **Control Block (`policy_engine.py`, `remediation_engine.py`, `k8s_client.py`):** Authorises, executes, verifies and records actions.
4. **Presentation Block (`frontend/src`):** Shows telemetry, incidents, live model outputs, remediation and the copilot, and provides the simulation controls and a role selector.

The main user actions are starting, pausing and resetting a failure scenario, choosing the target service, intensity and duration, opening an incident to run the RCA, asking the copilot a question, suggesting and executing a remediation, and switching the user role. Four roles are defined: Admin, DevOps, Developer and Viewer.

3.2.3 Dataset and Input Details

The experiments use synthetic datasets generated with fixed random seeds so that every run is repeatable. They are summarised in Table 3.1.

Dataset / Scenario	Entities	Features / Dimensions	Records	Type
Normal telemetry baseline	Healthy services of different size: CPU 5 to 45 %, memory 80 to 330 MB, latency 10 to 110 ms	5 features (CPU, memory, latency, error rate, pod restarts)	1000 samples, 5 % anomalous	Isolation Forest training
Failure data	Uniform operating ranges	7 features (adds historical failures and node CPU)	1500 samples, label by a weighted risk rule	Random Forest training
Health data	One third healthy ranges, two thirds uniform ranges	5 features, 3 classes by thresholds	1200 samples	Random Forest training
Simulated cluster	5 microservices, 2 nodes, 5 pods	CPU, memory, latency, error rate, replicas	6 failure scenarios	Simulation and RCA
Knowledge base	Runbooks and procedures	title, content, tags	4 documents	Retrieval
Deployments, pipelines, costs	v2.1.0 checkout, v1.4.2 payment, 2 pipeline runs, 4 cloud cost lines	version, build time, tests, daily cost	8 records	RCA, CI/CD and cost pages

Table 3.1: Sample Dataset Used for Experiment

The failure label is 1 when the weighted score of Equation (3.3) exceeds 0.48; this is the rule the Random Forest has to learn.

$$y = 1 \text{ if } 0.35\,(cpu/100) + 0.20\,(memory/1024) + 0.15\,(latency/1500) + 0.20\,(error/0.5) + 0.10\,(restarts/10) > 0.48 \quad (3.3)$$

3.2.4 Hardware and Software Specifications

The experiments and the end-to-end tests were executed on a computer system with the hardware and software specifications given in Table 3.2 and Table 3.3 respectively.

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 (2.4 GHz Multi-Core or higher)
HDD / SSD	256 GB NVMe SSD
RAM	8 GB / 16 GB DDR4

Table 3.2: Hardware details

Category	Specification
Operating System	Windows 11
Programming Language	Python 3.11 (backend, ML), JavaScript with JSX (frontend)
Backend	FastAPI, Uvicorn, SQLAlchemy, Pydantic, python-jose, passlib
Machine Learning	scikit-learn, NumPy, pandas, joblib
Frontend	React 18, Vite 5, Tailwind CSS, Recharts
Database	SQLite (default), PostgreSQL / TimescaleDB (optional)
Run mode	DEMO_MODE=true: simulated cluster, template explanations

Category	Specification
Testing	pytest (18 unit tests), Playwright for the dashboard screenshots

Table 3.3: Software details

Chapter 4
Performance and Results

4.1 Dashboard and Module Results

Figure 4.1 shows the Overview page at the healthy baseline. All five services are healthy, there are no active incidents, the cluster status is HEALTHY and the telemetry stream is flat. The header shows the DEMO BUILD label and the state of the subsystems: Kubernetes SIMULATED, ML READY, SLM DEMO TEMPLATE and WebSocket LIVE.

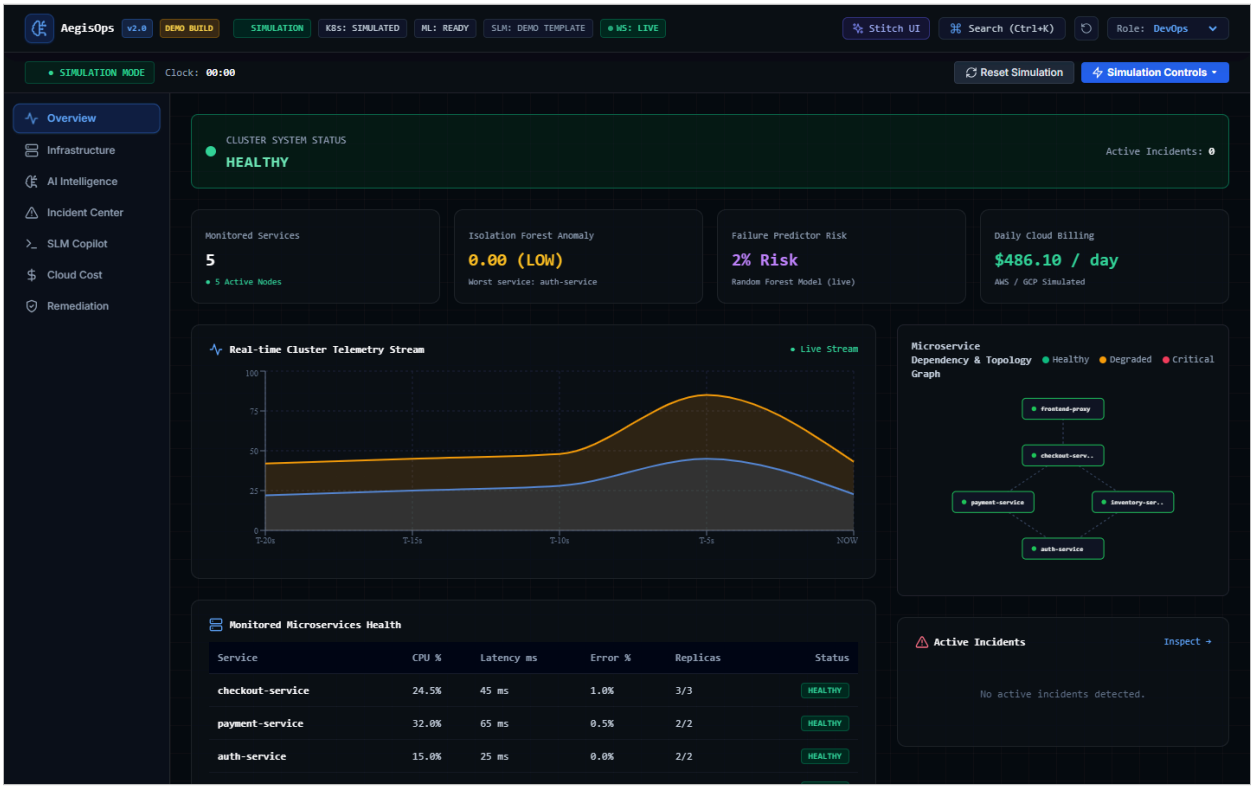


Fig 4.1: Overview page at the healthy baseline

Figure 4.2 shows the same page 19 simulated seconds after a CPU saturation scenario was started on the checkout service. The incident INC-1000 is opened by the Inject Scenario button, checkout-service shows 94.5 % CPU, 420 ms latency and 8.0 % errors with the status CRITICAL, and the cluster status is CRITICAL. The tiles now show live model outputs for the worst service: anomaly score 0.69 (HIGH) from the Isolation Forest and a failure risk of 36 % from the Random Forest.

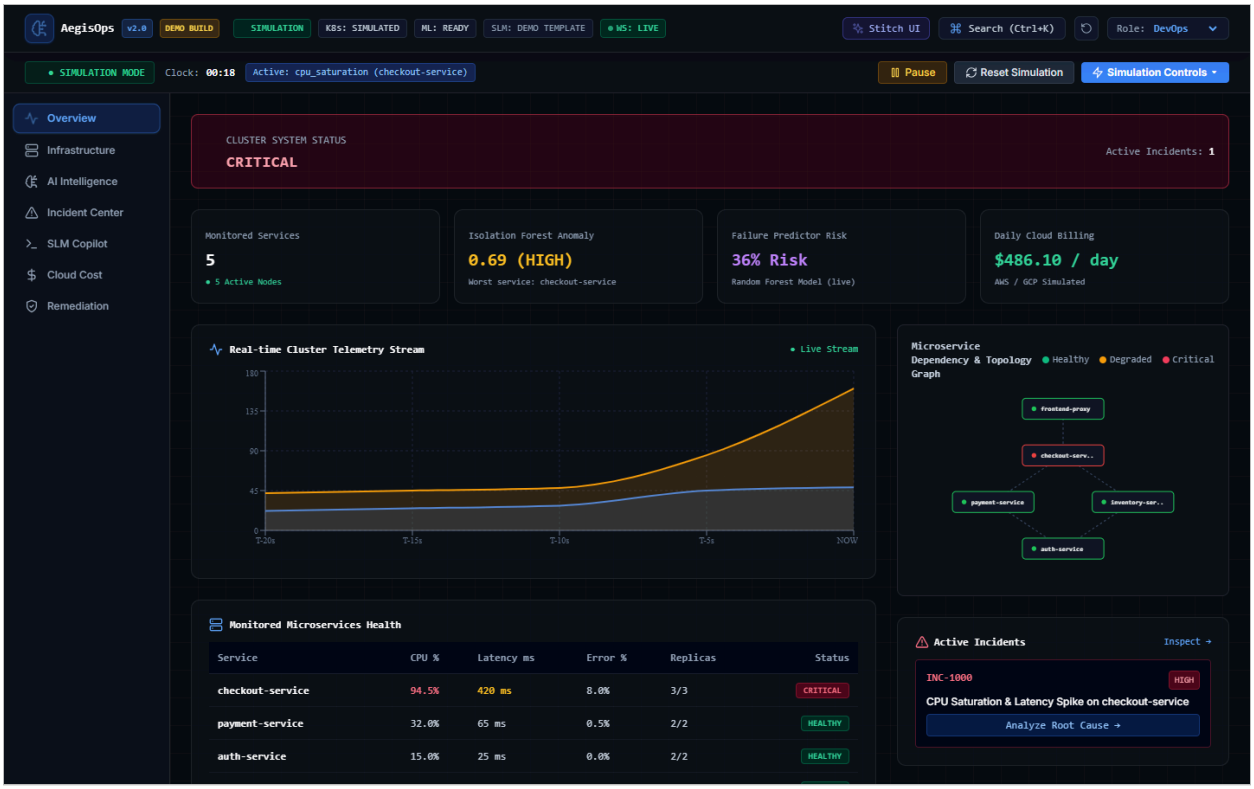


Fig 4.2: Overview page during a CPU saturation incident on checkout-service

Figure 4.3 shows the Infrastructure page with the two nodes (28.4 % and 31.2 % CPU) and the five pods with their nodes, restart counts and memory.

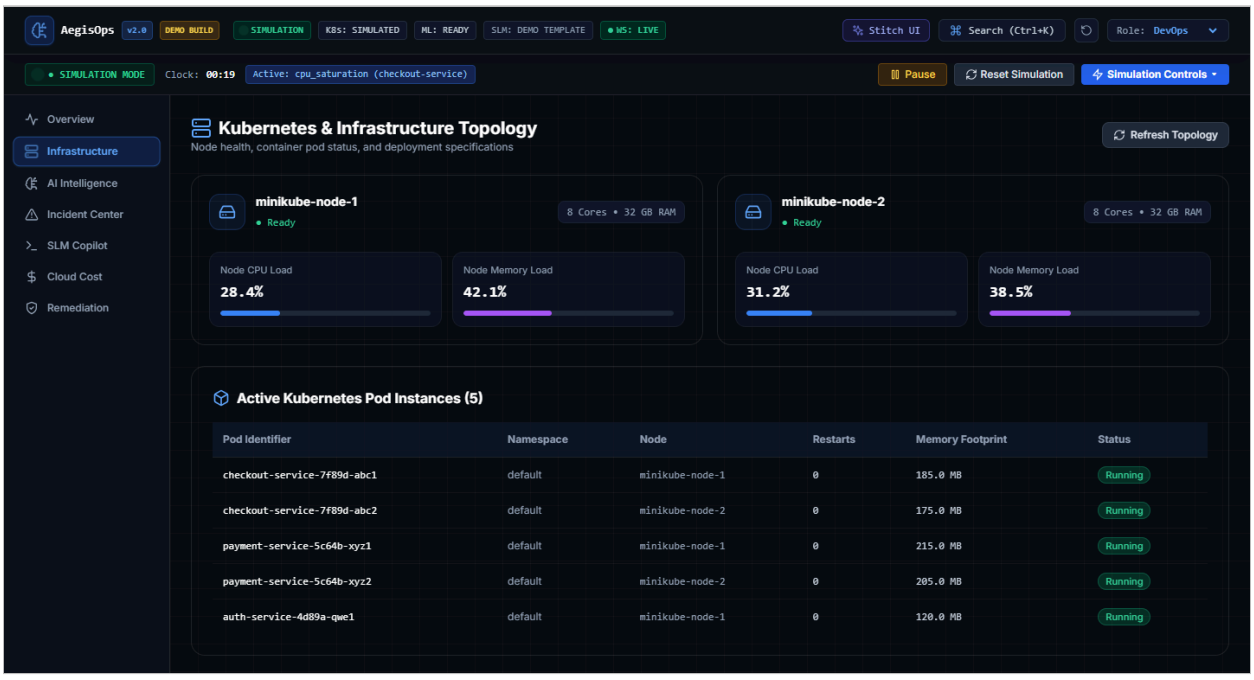


Fig 4.3: Infrastructure page: nodes and pods

Figure 4.4 shows the AI Intelligence page during the incident. The four model cards show the live scores for checkout-service: anomaly 0.69 (HIGH), failure probability 36 %, health class CRITICAL and fuzzy overload risk 100 / 100. The table lists every service (36.0 % MEDIUM for checkout-service and 0 to 2 % LOW for the others), and the fuzzy panel shows the memberships (CPU HIGH 1.00, latency MEDIUM 0.76) and the fired rule degrees (critical rule 1, high rule 0.24).

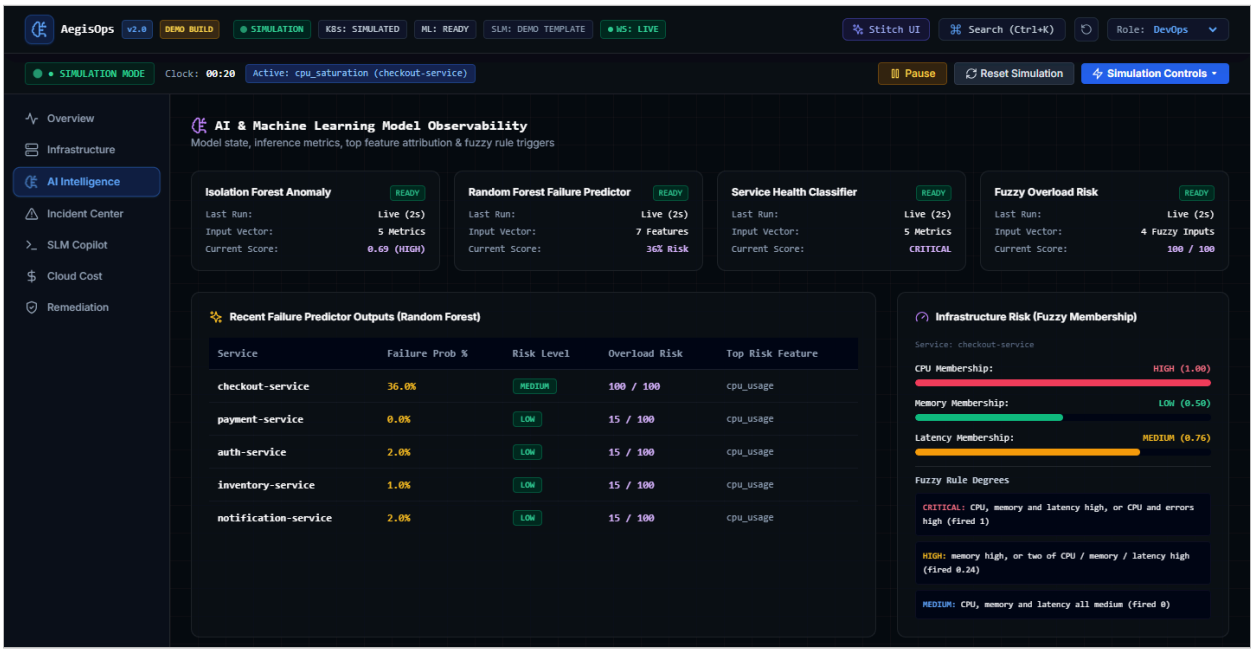


Fig 4.4: AI Intelligence page: live model outputs and fuzzy rule degrees

Figure 4.5 shows the Incident Center after selecting INC-1000. The probable root cause is CPU saturation of checkout-service with 88 % confidence, the evidence matrix is built from the incident telemetry (CPU 94.5 %, memory 180 MB, latency 420 ms, errors 8.0 % and no restarts), and the summary is produced by the demo template. The recent deployment v2.1.0 appears in the explanation as a contributing factor. The two retrieved runbooks are listed with their text.

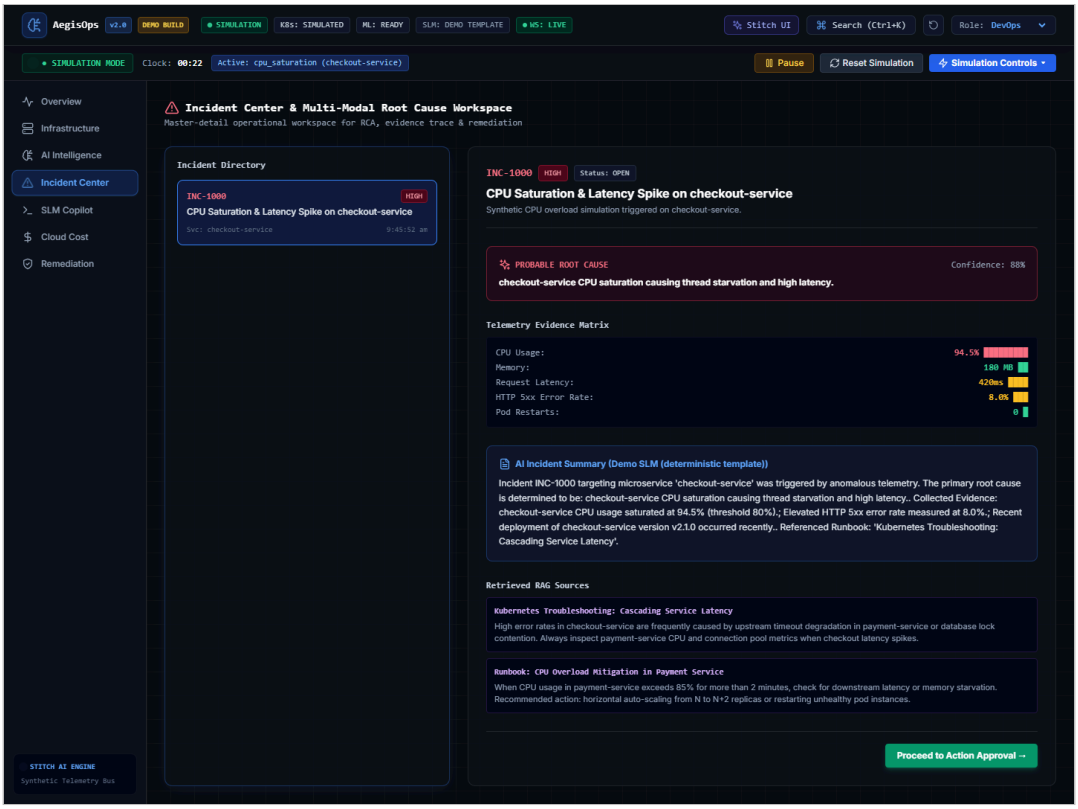


Fig 4.5: Incident Center: root cause, evidence, explanation and retrieved runbooks

Figure 4.6 shows the Remediation page with the Viewer role selected. The suggested SCALE_DEPLOYMENT action on checkout-service receives the policy decision DENY, its status is DENIED and the button is disabled.

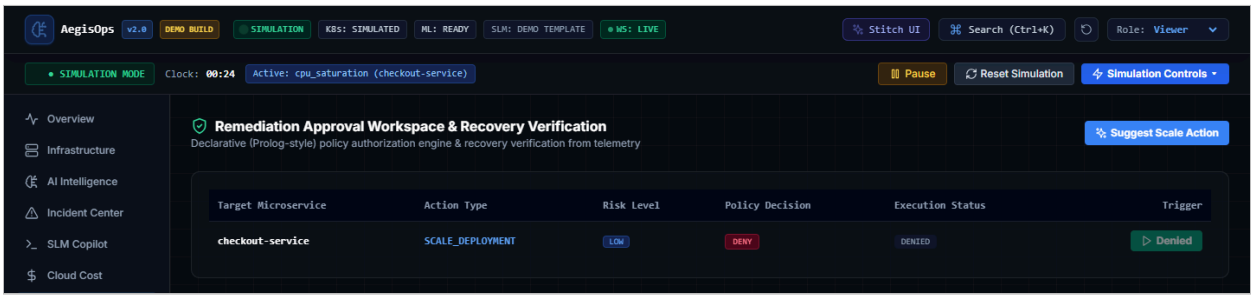


Fig 4.6: Remediation page for the Viewer role: action denied by policy

Figure 4.7 shows the page after the role was switched to DevOps. The same action now receives ALLOW, is executed and the banner reports the verification computed from the telemetry: CPU 37.8 %, memory 180 MB, latency 126 ms and error rate 0.1 %, all within healthy limits, and the status is SUCCESS.

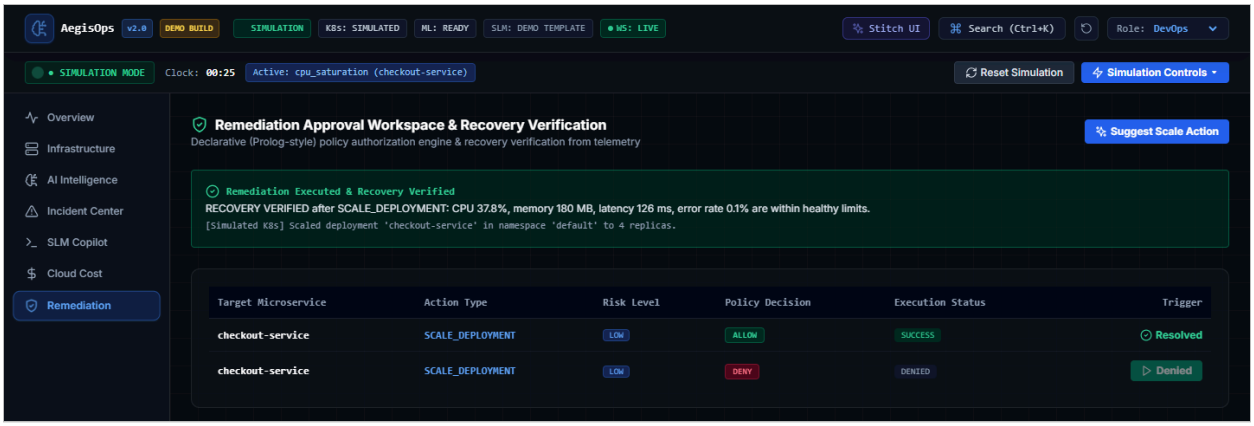


Fig 4.7: Remediation page for the DevOps role: executed and verified

Figure 4.8 shows the Overview page after the recovery. checkout-service is HEALTHY with 4 of 4 replicas, 37.8 % CPU and 126 ms latency, there are no active incidents, and the anomaly score is back to 0.00 (LOW).

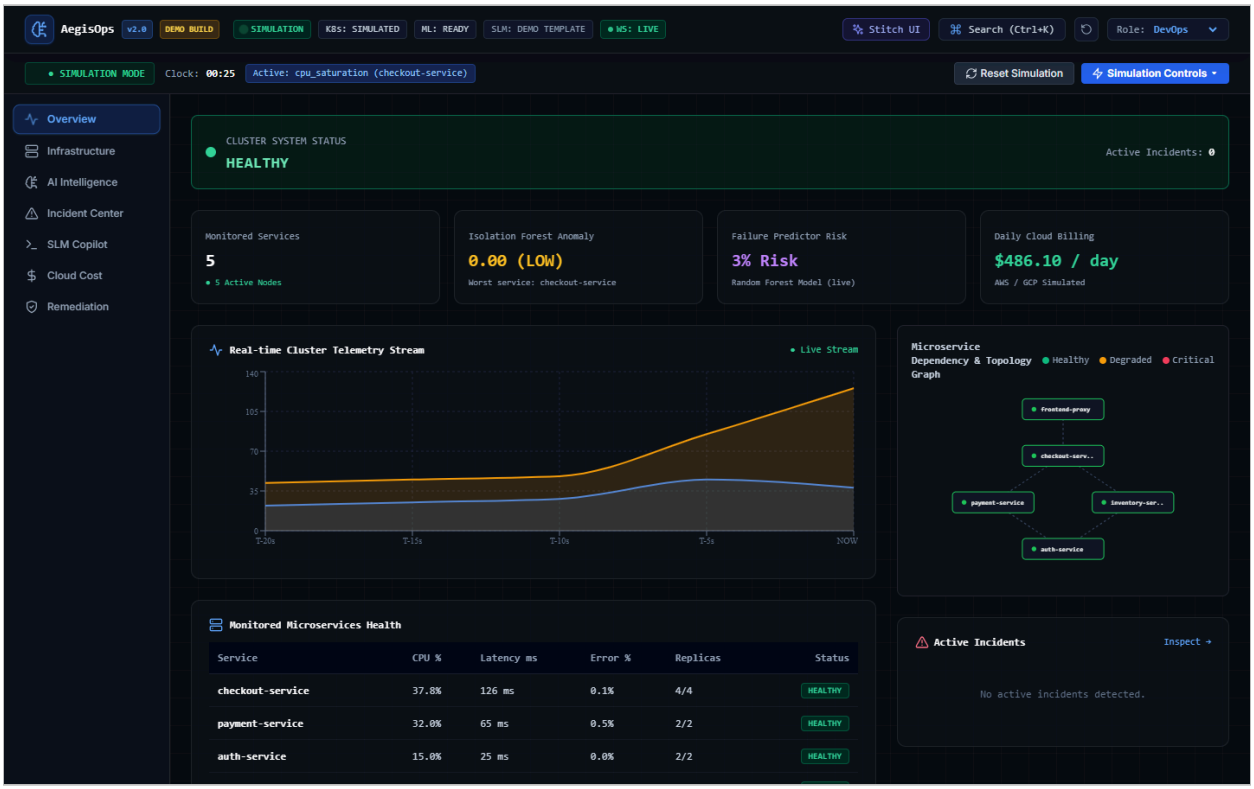


Fig 4.8: Overview page after verified recovery

Figure 4.9 shows the copilot. In demo mode the answer is built from the live system state: service health, active incidents, observability notes and the remediation history, with a note that it comes from a deterministic template and not from a language model.

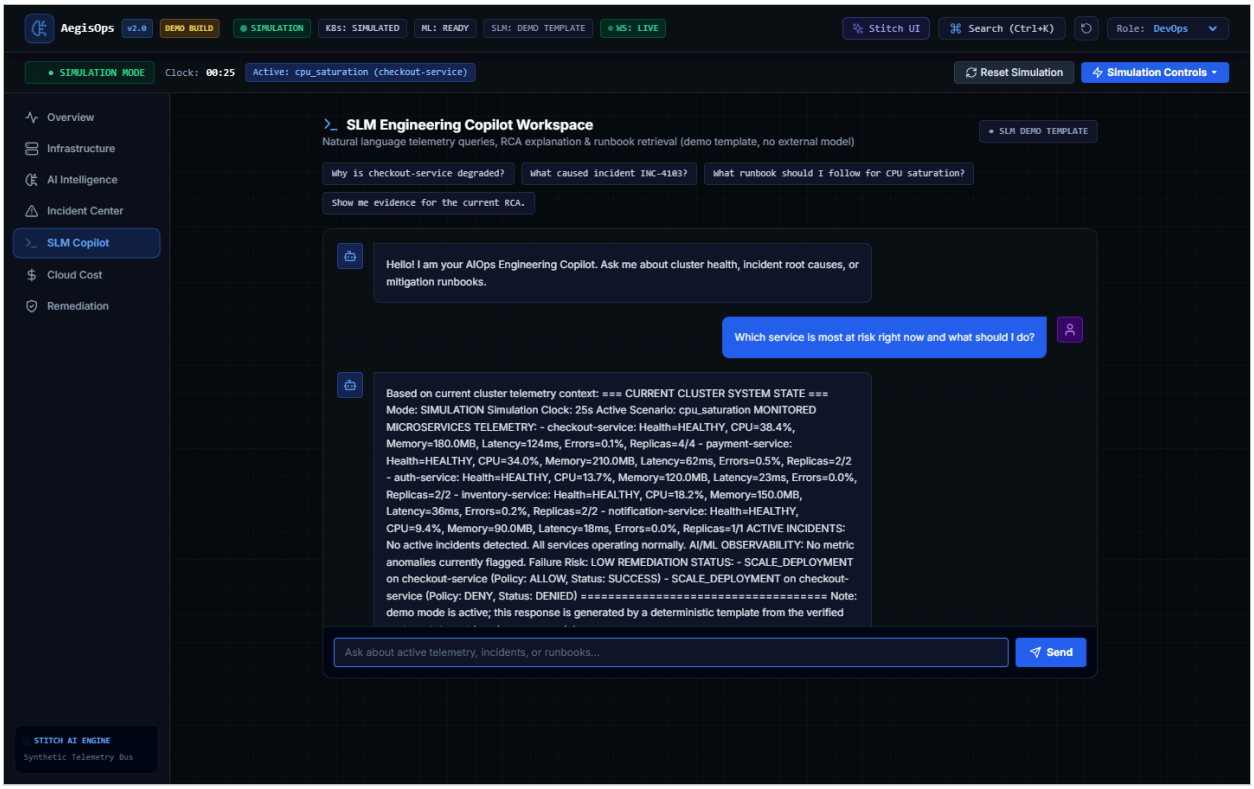


Fig 4.9: Copilot answering from live telemetry (demo template)

Figure 4.10 shows the Cloud Cost page with four simulated cost lines. The EC2 payment worker group at $240.00 per day exceeds the $200 per day rule and is flagged as an anomaly with a score of 0.96.

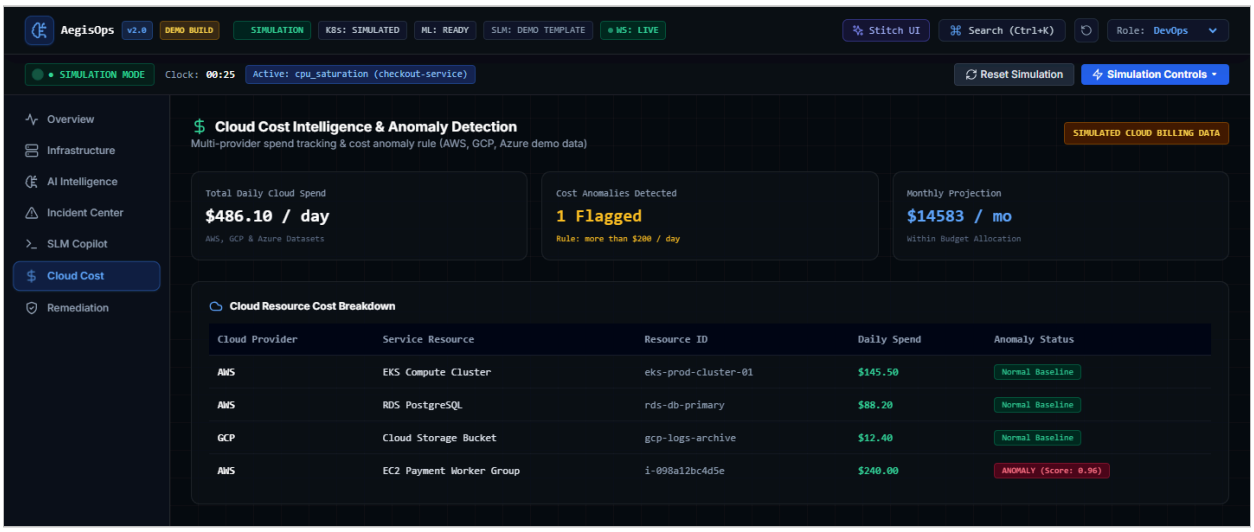


Fig 4.10: Cloud Cost page with an anomalous cost line

4.2 Comparative Results

Table 4.1 gives the results of the three trained models on new random draws (seeds different from the training seeds) of the same generators. The labels follow fixed rules, so these figures show that the models recover the rules.

Model	Test data	Accuracy	Precision	Recall	F1 score	ROC-AUC
Isolation Forest	950 normal and 50 anomalous	100 %	1.000	1.000	1.000	1.000
Random Forest failure predictor	1000 samples (486 failures)	94.1 %	0.942	0.936	0.939	0.989
Random Forest health classifier	1000 samples (337 healthy, 149 degraded, 514 critical)	99.9 %	0.999 (weighted)	0.999 (weighted)	0.999 (weighted)	-
Random Forest health classifier	999 balanced operating-range samples	100 %	-	-	-	-

Table 4.1: Performance of the machine learning models

For the failure predictor the confusion matrix on the 1000 held-out samples is 486 true normal, 28 false failures, 31 missed failures and 455 true failures, a false positive rate of 5.4 % and a false negative rate of 6.4 % (Figure 4.11). The most important features are CPU usage (0.477), error rate (0.172), memory (0.149) and latency (0.114), which follows the weights of the labelling rule in Equation (3.3). The Isolation Forest scores of the normal and anomalous samples do not overlap (Figure 4.12). The health classifier made one error in 1000 samples (a healthy sample labelled degraded) and none in 999 balanced samples from realistic operating ranges.

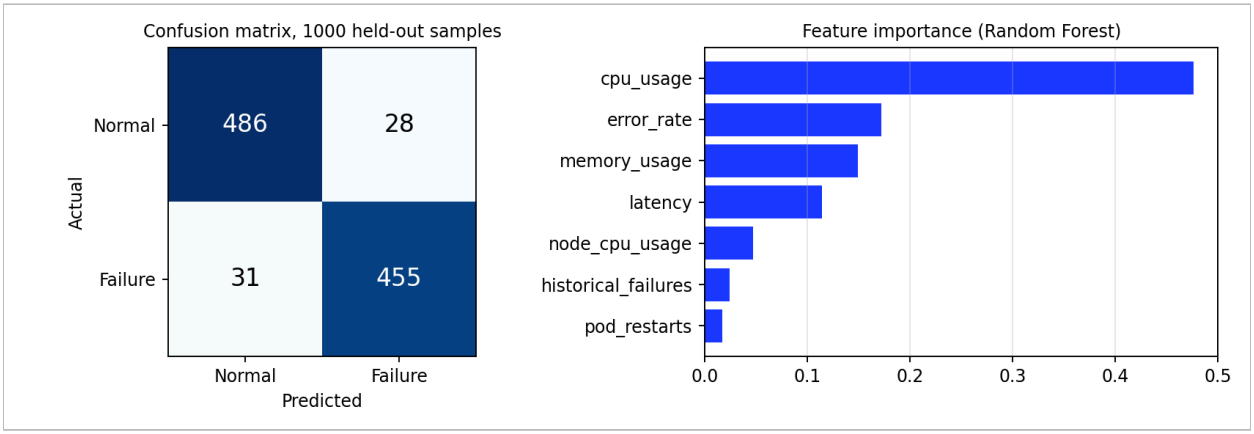


Fig 4.11: Failure predictor: confusion matrix and feature importance

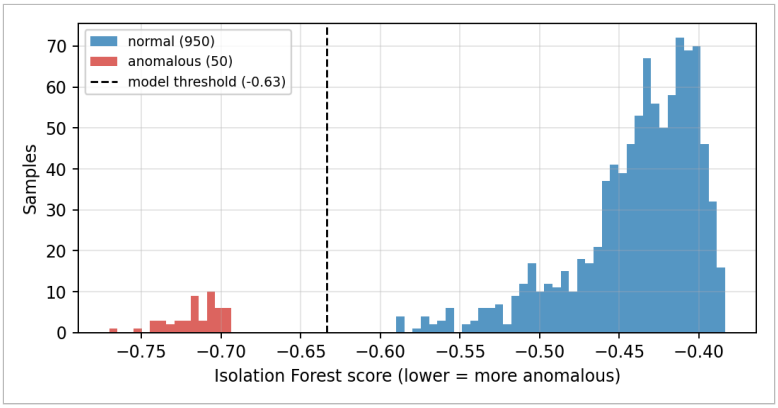


Fig 4.12: Isolation Forest scores of normal and anomalous telemetry

Table 4.2 shows the fuzzy overload-risk engine for typical situations. The score rises smoothly with load, and the memory-only fault and the crash loop are now rated HIGH because of the memory and restart rules.

Case	CPU %	Memory MB	Latency ms	Error %	Restarts	Score	Category
Idle service	15	150	30	0.1	0	0.150	LOW RISK
Moderate load	60	640	300	2	0	0.471	MEDIUM RISK
CPU saturation scenario	94.5	180	420	8	0	1.000	CRITICAL RISK
High load	90	800	650	10	0	1.000	CRITICAL RISK
Full saturation	98	950	1200	25	8	1.000	CRITICAL RISK
Memory-only leak	32	920	65	12	0	0.750	HIGH RISK
Pod crash loop	18	150	35	25	6	0.750	HIGH RISK

Table 4.2: Fuzzy overload-risk engine on test cases

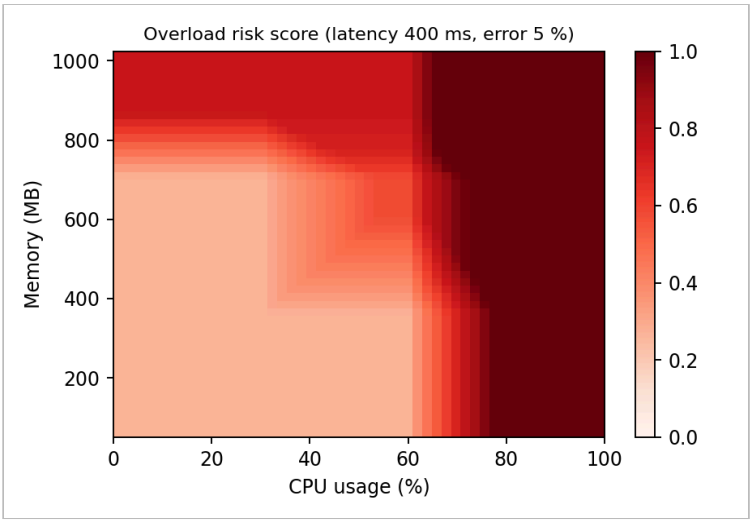


Fig 4.13: Overload risk score against CPU and memory

Table 4.3 shows the RCA engine on six test cases. Metric evidence (0.85 to 0.88) is ranked above a deployment that merely happened recently (0.75), which is reported as a contributing factor; a deployment becomes the primary cause (0.90) only when no metric explains the incident.

Case	Probable root cause reported	Confidence	Evidence items
CPU saturation, no deployment	checkout-service CPU saturation causing thread starvation	0.88	2
CPU saturation with deployment v2.1.0	checkout-service CPU saturation (deployment listed as a contributing factor)	0.88	3
Memory leak, 3 restarts	Memory leak or OOMKilled termination in payment-service	0.85	2
Crash loop, 9 restarts	Pod instability: repeated crashes (CrashLoopBackOff) in inventory-service	0.86	2

Case	Probable root cause reported	Confidence	Evidence items
Deployment of auth-service (a dependency of checkout)	Recent code deployment (v3.0.1) of auth-service	0.90	2
Healthy service	Transient network jitter or unclassified load anomaly	0.60	0

Table 4.3: Root cause analysis on test cases

Table 4.4 summarises the policy engine over all 32 combinations of four roles, four action types and two risk levels. The engine is deterministic: 7 combinations are allowed, 8 need approval and 17 are denied. Deleting a resource on a protected target (database-primary or kube-system) is denied even for an Admin, an unknown role is treated as Viewer and an unknown risk level requires approval.

Role	ALLOW	REQUIRE_APPROVAL	DENY	Example rule
Admin	5	2	1	rule(Admin, SCALE_DEPLOYMENT, HIGH) -> ALLOW
DevOps	2	4	2	rule(DevOps, SCALE_DEPLOYMENT, LOW) -> ALLOW
Developer	0	2	6	rule(Developer, RESTART_POD, LOW) -> REQUIRE_APPROVAL
Viewer	0	0	8	rule(Viewer, SCALE_DEPLOYMENT, LOW) -> DENY
Total	7	8	17	

Table 4.4: Policy decisions for 32 role, action and risk combinations

Table 4.5 gives the results of the end-to-end incident tests run through the REST API on a freshly seeded database. In every scenario the incident was opened, the RCA found the cause, the suggested action received the policy decision ALLOW (or REQUIRE_APPROVAL for the rollback), and the recovery was verified from the telemetry before the incident was resolved. The role checks behaved as designed: a Viewer received DENY and the execution was refused, a Developer received REQUIRE_APPROVAL and a Viewer could not approve the action, but a DevOps user could.

Scenario	Anomaly (score)	Failure prob.	Fuzzy risk	Root cause (confidence)	Action and policy	Outcome
CPU spike, checkout-service	HIGH (0.69)	0.36 MEDIUM	1.00 CRITICAL	CPU saturation (0.88); deployment v2.1.0 contributing	SCALE_DEPLOYMENT, ALLOW	CPU 94.5 to 37.8 %, latency 420 to 126 ms, errors 8 to 0.1 %; RESOLVED
Memory leak, payment-service	CRITICAL (1.00)	0.08 LOW	0.75 HIGH	Memory leak or OOM (0.85); deployment v1.4.2 contributing	RESTART_POD, ALLOW	Memory 920 to 180 MB, errors 12 to 0.1 %; RESOLVED

Scenario	Anomaly (score)	Failure prob.	Fuzzy risk	Root cause (confidence)	Action and policy	Outcome
Crash loop, inventory-service	CRITICAL (1.00)	0.07 LOW	0.75 HIGH	CrashLoopBackOff (0.86)	RESTART_POD, ALLOW	Errors 25 to 0.1 %; RESOLVED
CPU spike, rollback chosen	-	-	-	-	ROLLBACK_DEPLOYMENT, REQUIRE_APPROVAL (HIGH risk)	Developer refused; DevOps approved: CPU 94.5 to 42.5 %, latency 420 to 147 ms; SUCCESS

Table 4.5: End-to-end incident tests

The failure predictor gives a low probability for the memory leak and the crash loop (0.08 and 0.07) because its label rule is dominated by CPU and error rate; these faults are caught by the Isolation Forest, the fuzzy engine and the RCA engine instead, which shows why several complementary models are used.

Table 4.6 gives measured times on the test computer. The models answer in about 5 to 10 ms per call, the fuzzy engine in microseconds, and a whole incident from trigger to verified recovery, with six REST calls, takes about 0.13 s on the server.

Operation	Time
Isolation Forest, one sample	10.5 ms
Failure predictor, one sample	9.9 ms
Health classifier, one sample	4.7 ms
Fuzzy overload-risk engine, one sample	0.009 ms
Analysis endpoint (four models, one service)	47 ms
Incident analysis endpoint (RCA, retrieval, explanation)	14 ms
Remediation execution (rollback, with verification)	33 ms
One full incident, trigger to resolved (6 REST calls)	0.12 to 0.13 s
Unit tests (18 tests)	5.1 s

Table 4.6: Measured response times

4.3 Problems Found and Fixed During Testing

Testing the first version of the system showed several problems. They were fixed, covered by unit tests and re-measured, and the results above are those of the fixed version. Table 4.7 lists them.

Problem found	Fix
The anomaly score 1 − (s + 0.5) put normal telemetry near 0.85, above the 0.65 alert level, so every sample was reported as a CRITICAL anomaly.	The decision now comes from the Isolation Forest itself and the score is calibrated (Equation 3.1): normal 0.00, abnormal 0.5 to 1.0.
Small healthy services (for example a notification service with 90 MB of memory) looked abnormal because the baseline data had one narrow normal range.	The baseline now covers healthy services of different sizes (Table 3.1).
The health classifier had only 0.5 % healthy samples in its training data.	One third of the training samples now come from healthy ranges.
The fuzzy rules ignored memory-only faults and crash loops (score 0.15).	Rules for high memory, latency with errors, and restarts with errors were added (score 0.75).
A recent deployment outranked the metric evidence, so a CPU or memory fault always suggested a rollback; a crash loop was called a memory leak.	Metric evidence now ranks first, the deployment is a contributing factor, and a crash-loop cause with a restart action was added.
The executor did not implement ROLLBACK_DEPLOYMENT, did not refuse denied actions and did not check who approved an action.	Rollback was added; denied actions are refused; approval needs an Admin or DevOps role.
The create endpoint ignored the role and risk, so the role selector had no effect; recovery verification was a fixed message.	The role and risk are sent with the request, and the verification is computed from the telemetry after the action.
The Inject Scenario button did not open an incident, the dashboard tiles (0.91, 87 %, $486.10) were fixed values, and two incidents created in the same second got the same code.	The button opens the incident, the tiles show live model outputs and the summed cost, and incident codes are unique.
Calls to an Ollama server that was not running made each explanation wait for failed connections.	Demo mode answers from the template immediately; with Ollama enabled a failed call is skipped for 60 s.
The stored model files came from a newer scikit-learn version and the cost page named the Isolation Forest for a simple rule.	The models were retrained and the cost page now states the rule (more than $200 per day).

Table 4.7: Problems found during testing and their fixes

Remaining limits. The data and the cluster are simulated, and the labels follow fixed rules, so the accuracies are not estimates of real-world performance. Recovery is simulated by scaling the metrics (for example CPU multiplied by 0.4) and then verified by re-reading them. The failure predictor does not respond to a memory leak or a crash loop on their own. The policy engine is a Prolog-style rule table written in Python, runbook retrieval uses word overlap, the cost anomaly is a threshold rule, and the language model was not used in the demo build.

Chapter 5
Applications

5.1 Social

There are various applications of the AegisOps demonstration in education and in thinking about the reliability of public digital services. The primary applications are listed below under Social and Technical categories.

5.1.1 Learning AIOps Safely

Students can inject a CPU spike, a memory leak or a crash loop, watch the models react, read the evidence, switch the user role and see an action allowed, held for approval or denied, without risking a real production system.

5.1.2 Reliable Public Digital Services

Banking, healthcare, transport and government portals run on the same kind of container platforms. The workflow shown here, detect early, find the cause, act under policy and verify, indicates how the time that citizens cannot book an appointment or pay a bill could be reduced.

5.1.3 Reduced On-Call Fatigue

Engineers who are woken at night for every threshold alert lose sleep and attention. Ranked causes, plain-language explanations and policy-approved fixes would let a person handle only the cases that need judgement.

5.2 Technical

5.2.1 Incident Response and Root Cause Analysis

The RCA engine combines metric, deployment and dependency evidence in one ranked list, which shortens the search that engineers do by hand. The dependency graph shows how a fault in payment-service can propagate to checkout-service.

5.2.2 Predictive Operations and CI/CD Risk

The same failure predictor is applied to pipeline runs (build time and failed tests) to estimate the risk that a build will fail. Failure probabilities could drive predictive autoscaling before the saturation point is reached.

5.2.3 Governed Automation and Cost Awareness

The policy engine, the approval rule and the audit log show the governance that organisations need before they allow automatic changes. The cost page shows how spend per cloud resource can be monitored for anomalies in the same console.

Chapter 6
Conclusion and Future Scope

6.1 Conclusion

In this project we designed, implemented and evaluated AegisOps, a demonstration system for AI-powered monitoring and self-healing of cloud and Kubernetes workloads. The system observes the telemetry of five simulated microservices, detects anomalies with an Isolation Forest, predicts failure with a Random Forest (94.1 % accuracy and 0.989 ROC-AUC on held-out synthetic data), classifies health (99.9 %), scores overload risk with a fuzzy engine, ranks root causes from metric, deployment and dependency evidence, retrieves runbooks for the explanation, and authorises every action with a deterministic policy table before scaling, restarting or rolling back in simulation. In the end-to-end tests the CPU, memory and crash-loop incidents were resolved with verified recovery, the role rules were enforced, and all 18 unit tests pass.

Testing also exposed problems in the first version, such as an uncalibrated anomaly score, rules that missed memory faults, a deployment that outranked the evidence, a missing rollback and fixed dashboard values. They were fixed and are listed in Section 4.3 so that the results can be read correctly. The project shows how classical machine learning, fuzzy reasoning, rule-based policy and runbook retrieval can be combined into one explainable operations workflow in which the AI proposes and the policy decides.

6.2 Future Scope

The following extensions are planned:

1. **Real Telemetry and Verification:** Read metrics from Prometheus, train the models on real incidents, and verify recovery from the real telemetry after an action instead of a simulated update.
2. **Live Cluster Actions:** Wire the rollback action to the Kubernetes API and test scale and restart on Minikube or kind.
3. **Learned Fuzzy Memberships:** Tune the membership functions and rule weights from data (a neuro-fuzzy system).
4. **Prolog Policy Engine and Vector Retrieval:** Run the policy as a real Prolog program (for example in SWI-Prolog) and use a vector store for runbook retrieval.
5. **Language Model Evaluation:** Turn demo mode off, run a small model through Ollama, and compare the quality and speed of its explanations with the template.
6. **Better Failure Prediction:** Add memory growth and restart features to the failure predictor so that it also anticipates memory leaks and crash loops.

References

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[2] F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation forest,” in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.

[3] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[4] L. A. Zadeh, “Fuzzy sets,” *Information and Control*, vol. 8, no. 3, pp. 338–353, June 1965.

[5] E. H. Mamdani and S. Assilian, “An experiment in linguistic synthesis with a fuzzy logic controller,” *International Journal of Man-Machine Studies*, vol. 7, no. 1, pp. 1–13, 1975.

[6] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020, pp. 9459–9474.

[7] R. A. Kowalski, “Predicate logic as programming language,” in *Proc. IFIP Congress*, Stockholm, Sweden, 1974, pp. 569–574.

[8] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[9] B. Beyer, C. Jones, J. Petoff and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA, USA: O’Reilly Media, 2016.

[10] P. Notaro, J. Cardoso and M. Gerndt, “A survey of AIOps methods for failure management,” *ACM Transactions on Intelligent Systems and Technology*, vol. 12, no. 6, Art. 81, 2021.

[11] B. Burns, B. Grant, D. Oppenheimer, E. Brewer and J. Wilkes, “Borg, Omega, and Kubernetes,” *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016.

[12] Qwen Team, “Qwen2.5 technical report,” arXiv:2412.15115, 2024.

Acknowledgement

We would like to express our special thanks to Prof Nirosha Uppu, our mini project guide who guided us through the project and who helped us in applying the knowledge that we have acquired during the semester and learning new concepts.

We would like to express our special thanks to Dr. Shankar M. Patil, Head, Department of Artificial Intelligence and Data Science, who gave us the opportunity to do this mini project because of which we learned new concepts and their application.

We are also thankful to our mini project coordinator Prof Nirosha Uppu along with other faculties for their encouragement and support.

Finally we would like to express our special thanks of gratitude to Principal Dr. Sunil Chavan who gave us the opportunity and facilities to conduct this mini project.

Nihal Mishra